AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 15: Reduction to Hessenberg and Tridiagonal Forms; Rayleigh Quotient Iteration

Prof. Xiangmin Jiao

Stony Brook University

Outline

1. **Schur Factorization (NLA§26)**
2. Reduction to Hessenberg and Tridiagonal Forms (NLA§26)
3. Rayleigh Quotient Iteration (NLA§27)

“Obvious” Algorithms

- The most obvious method is to find roots of the characteristic polynomial $p_A(\lambda)$, but it is very ill-conditioned.
- Another idea is power iteration, using the fact that

$$\frac{x}{\|x\|}, \frac{Ax}{\|Ax\|}, \frac{A^2x}{\|A^2x\|}, \frac{A^3x}{\|A^3x\|}, \cdots$$

  converges to an eigenvector corresponding to the largest eigenvalue of A in absolute value, but it may converge very slowly.
- Instead, we can compute an eigenvalue-revealing factorization, such as the Schur factorization

$$A = QTQ^*$$

  by introducing zeros, using algorithms similar to QR factorization.

A Fundamental Difficulty

- However, an eigenvalue-revealing factorization cannot be done in a finite number of steps.

  Any general eigenvalue solver must be iterative.

- To see this, consider a general polynomial of degree n

$$p(z) = z^n + a_{n-1}z^{n-1} + \cdots + a_1 z + a_0.$$

  There is no closed-form expression for roots when $n > 4$. In general, the roots of polynomial equations higher than fourth degree cannot be written in terms of a finite number of elementary arithmetic operations and root extractions (Abel–Ruffini theorem, 1824).

A Fundamental Difficulty (Cont'd)

- However, the roots of p_A are the eigenvalues of the *companion matrix*

$$A = \begin{bmatrix} 0 & & & & -a_0 \\ 1 & 0 & & & -a_1 \\ & 1 & \ddots & & \vdots \\ & & \ddots & 0 & -a_{n-2} \\ & & & 1 & -a_{n-1} \end{bmatrix}.$$

- Therefore, in general, we cannot find the eigenvalues of a matrix in a finite number of steps.
- In practice, however, there are algorithms that converge to the desired precision in a few iterations.

Schur Factorization and Diagonalization

- Most eigenvalue algorithms compute the Schur factorization $A = QTQ^*$ by transforming A with similarity transformations

$$\underbrace{Q_j^* \cdots Q_2^* Q_1^*}_{Q^*} A \underbrace{Q_1 Q_2 \cdots Q_j}_{Q},$$

  where Q_i are unitary matrices, which converge to T as $j \to \infty$.
- Note that real matrices might need complex Schur forms and eigenvalues.
- Question: For a Hermitian A, what matrix will the sequence converge to, assuming eigenvalues are distinct in magnitude?

Outline

1. Schur Factorization (NLA§26)
2. **Reduction to Hessenberg and Tridiagonal Forms (NLA§26)**
3. Rayleigh Quotient Iteration (NLA§27)

Two Phases of Eigenvalue Computations

- For a general matrix, first make it *upper-Hessenberg* and then upper triangular:

$$\underset{A\neq A^*}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}} \xrightarrow{\text{Phase 1}} \underset{\text{upper-Hessenberg}}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}} \xrightarrow{\text{Phase 2}} \underset{\text{triangular}}{\begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \\ & & & & \times \end{bmatrix}}$$

- For a Hermitian matrix, first *tridiagonal* form and then diagonal:

$$\underset{A=A^*}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}} \xrightarrow{\text{Phase 1}} \underset{\text{tridiagonal}}{\begin{bmatrix} \times & \times & & & \\ \times & \times & \times & & \\ & \times & \times & \times & \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}} \xrightarrow{\text{Phase 2}} \underset{\text{diagonal}}{\begin{bmatrix} \times & & & & \\ & \times & & & \\ & & \times & & \\ & & & \times & \\ & & & & \times \end{bmatrix}}$$

- Phase 1 is direct and requires $\mathcal{O}\left(n^3\right)$ flops. Phase 2 is iterative and requires $\mathcal{O}\left(n\right)$ iterations and $\mathcal{O}\left(n^2\right)$ flops for Hermitian matrices ($\mathcal{O}\left(n^3\right)$ flops for non-Hermitian).

Introducing Zeros by Similarity Transformations

- The first attempt is to compute the Schur factorization $A = QTQ^*$ by applying Householder reflectors from both left and right

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}} \xrightarrow{Q_1^* \cdot} \underset{Q_1^* A}{\begin{bmatrix} \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \end{bmatrix}} \xrightarrow{\cdot Q_1} \underset{Q_1^* A Q_1}{\begin{bmatrix} \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \end{bmatrix}}$$

- Unfortunately, the right multiplication destroys the zeros introduced by Q_1^*.
- This would not work because of Abel's theorem.
- However, the subdiagonal entries typically decrease in magnitude.

The Hessenberg Form

- The second attempt is to try to compute an upper Hessenberg matrix H similar to A

$$\underset{A}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix}} \xrightarrow{Q_1^* \cdot} \underset{Q_1^* A}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \end{bmatrix}} \xrightarrow{\cdot Q_1} \underset{Q_1^* A Q_1}{\begin{bmatrix} \times & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \times & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \end{bmatrix}}$$

- The zeros introduced by $Q_1^* A$ were not destroyed this time!
- Continuing with the remaining columns would result in the Hessenberg form

$$\xrightarrow{Q_2^* \cdot} \underset{Q_2^* Q_1^* A Q_1}{\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \mathbf{x} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & \mathbf{0} & \mathbf{x} & \mathbf{x} & \mathbf{x} \end{bmatrix}} \xrightarrow{\cdot Q_2} \underset{Q_2^* Q_1^* A Q_1 Q_2}{\begin{bmatrix} \times & \times & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ \times & \times & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & \times & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & & \mathbf{x} & \mathbf{x} & \mathbf{x} \\ & & \mathbf{x} & \mathbf{x} & \mathbf{x} \end{bmatrix}} \cdots$$

The Hessenberg Form

- After $n-2$ steps, we obtain the Hessenberg form

$$\underbrace{Q_{n-2}^* \cdots Q_2^* Q_1^*}_{Q^*} A \underbrace{Q_1 Q_2 \cdots Q_{n-2}}_{Q} = H = \begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}.$$

- For a Hermitian matrix A, H is Hermitian and hence is tridiagonal.

Householder Reduction to Hessenberg

Algorithm: Householder Reduction to Hessenberg Form

for $k = 1, \dots, n-2$ **do**

$x = A_{k+1:n,k}$

$v_k = \text{sign}(x_1)\|x\|_2 e_1 + x$

$v_k = v_k / \|v_k\|_2$

$A_{k+1:n,k:n} = A_{k+1:n,k:n} - 2v_k(v_k^* A_{k+1:n,k:n})$

$A_{1:n,k+1:n} = A_{1:n,k+1:n} - 2(A_{1:n,k+1:n} v_k) v_k^*$

end for

- Note that Q is never formed explicitly.
- The operation count is

$$\sim \sum_{k=1}^{n-2} [4(n-k)^2 + 4n(n-k)] = \frac{10}{3} n^3.$$

Reduction to Tridiagonal Form

- If A is Hermitian, then

$$\underbrace{Q_{n-2}^* \cdots Q_2^* Q_1^*}_{Q^*} A \underbrace{Q_1 Q_2 \cdots Q_{n-2}}_{Q} = H = \begin{bmatrix} \times & \times & & & \\ \times & \times & \times & & \\ & \ddots & \ddots & \ddots & \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}.$$

- For a Hermitian A, the operation count would be the same as Householder QR, which is $\frac{4}{3}n^3$.
  - First, taking advantage of sparsity, the cost of applying right reflectors is also $4(n-k)^2$ instead of $4n(n-k)$, so the cost is

$$\sim \sum_{k=1}^{n-2} 8(n-k)^2 = \frac{8}{3}n^3.$$

  - Second, taking advantage of symmetry, the cost is reduced by 50% to $\frac{4}{3}n^3$.

Stability of Hessenberg Reduction

Theorem

Householder reduction to Hessenberg form is backward stable, in that

$$\tilde{Q}\tilde{H}\tilde{Q}^* = A + \delta A, \quad \frac{\|\delta A\|}{\|A\|} = \mathcal{O}\left(\epsilon_{\text{machine}}\right)$$

for some $\delta A \in \mathbb{C}^{n\times n}$.

Note that, similar to Householder QR, $\tilde{Q}$ is exactly unitary based on some $\tilde{v}_k$.

Outline

1. Schur Factorization (NLA§26)
2. Reduction to Hessenberg and Tridiagonal Forms (NLA§26)
3. **Rayleigh Quotient Iteration (NLA§27)**

Solving Eigenvalue Problems

- All eigenvalue solvers must be iterative.
- Iterative algorithms have multiple facets.
  1. Basic idea behind the algorithms
  2. Convergence and techniques to speed up convergence
  3. Efficiency of implementation
  4. Termination criteria
- We will focus on the first two aspects.

Simplification: Real Symmetric Matrices

- We will consider eigenvalue problems for real symmetric matrices, i.e., $A = A^\top \in \mathbb{R}^{n\times n}$, and $Ax = \lambda x$ for $x \in \mathbb{R}^n$.
  - Note that $x^* = x^\top$, and $\|x\| = \sqrt{x^\top x}$.
- A has real eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ and orthonormal eigenvectors $q_1, q_2, \dots, q_n$, where $\|q_j\| = 1$.
- Eigenvalues are often also ordered in a particular way (e.g., ordered from large to small in magnitude).
- In addition, we focus on symmetric tridiagonal form.
  - Why? Because phase 1 of the two-phase algorithm reduces the matrix into tridiagonal form.

Rayleigh Quotient

- The Rayleigh quotient of $x \in \mathbb{R}^n$ is the scalar

$$r(x) = \frac{x^\top A x}{x^\top x}.$$

- For an eigenvector x, its Rayleigh quotient is $r(x) = \frac{x^\top \lambda x}{x^\top x} = \lambda$, which is the corresponding eigenvalue of x.
- For a general x, $r(x) = \alpha$ that minimizes $\|Ax - \alpha x\|_2$.
- x is an eigenvector of $A \iff \nabla r(x) = \frac{2}{x^\top x}(Ax - r(x)x) = 0$ with $x \neq 0$.
- $r(x)$ is smooth and $\nabla r(q_j) = 0$ for any j, and therefore is quadratically accurate

$$|r(x) - r(q_J)| = \mathcal{O}\left(\|x - q_J\|^2\right) \text{ as } x \to q_J \text{ for some } J.$$

Power Iteration

- A simple power iteration for the largest eigenvalue.

Algorithm: Power Iteration

$v^{(0)} =$ some unit-length vector
for $k = 1, 2, \ldots$ **do**
  $w = Av^{(k-1)}$
  $v^{(k)} = w/\|w\|$
  $\lambda^{(k)} = r(v^{(k)}) = (v^{(k)})^{\mathsf{T}} A v^{(k)}$
end for

- The termination condition is omitted for simplicity.

Convergence of Power Iteration

- Expand the initial $v^{(0)}$ in orthonormal eigenvectors q_i, and apply A^k

$$
\begin{aligned}
v^{(0)} &= a_1 q_1 + a_2 q_2 + \cdots + a_n q_n \\
v^{(k)} &= c_k A^k v^{(0)} = c_k (a_1 \lambda_1^k q_1 + a_2 \lambda_2^k q_2 + \cdots + a_n \lambda_n^k q_n) \\
&= c_k \lambda_1^k \left(a_1 q_1 + a_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k q_2 + \cdots + a_n \left(\frac{\lambda_n}{\lambda_1}\right)^k q_n \right).
\end{aligned}
$$

- Assume A has a unique dominant eigenvalue such that $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n| \geq 0$. If $q_1^\top v^{(0)} \neq 0$, this gives

$$
\|v^{(k)} - (\pm q_1)\| = \mathcal{O}\left(\left| \frac{\lambda_2}{\lambda_1} \right|^k \right), \quad |\lambda^{(k)} - \lambda_1| = \mathcal{O}\left(\left| \frac{\lambda_2}{\lambda_1} \right|^{2k} \right)
$$

  where the $\pm$ sign is chosen to be the sign of $q_1^\top v^{(k)}$.
- It finds the largest eigenvalue (unless the eigenvector is orthogonal to $v^{(0)}$).
- The error reduces by only a constant factor ($\approx |\lambda_2/\lambda_1|$) each step, and very slowly especially when $|\lambda_2| \approx |\lambda_1|$.

Inverse Iteration

- Apply power iteration on $(A - \mu I)^{-1}$, with eigenvalues $\{(\lambda_j - \mu)^{-1}\}$.
- If μ is strictly closer to λ_J than to any other eigenvalue, then $|(\lambda_J - \mu)^{-1}|$ is strictly larger than $|(\lambda_j - \mu)^{-1}|$ for $j \neq J$, so the power iteration converges rapidly to q_J.

Algorithm: Inverse Iteration

$v^{(0)} =$ some unit-length vector
for $k = 1, 2, \ldots$ **do**
  Solve $(A - \mu I)w = v^{(k-1)}$ for w
  $v^{(k)} = w/\|w\|$
  $\lambda^{(k)} = r(v^{(k)}) = (v^{(k)})^{\mathsf{T}} A v^{(k)}$
end for

Convergence of Inverse Iteration

- It converges to the eigenvector q_J if the shift μ satisfies $|\mu - \lambda_J| < |\mu - \lambda_K| \leq |\mu - \lambda_j|$ for all $j \neq J, K$, with

$$\|v^{(k)} - (\pm q_J)\| = \mathcal{O}\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^k\right), \quad |\lambda^{(k)} - \lambda_J| = \mathcal{O}\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^{2k}\right)$$

  where λ_J and λ_K are the closest and second closest eigenvalues to μ.
- This is a standard method for determining an eigenvector given an eigenvalue estimate.

Rayleigh Quotient Iteration

- The parameter μ is constant in inverse iteration, but convergence is better for μ close to the eigenvalue.
- As an improvement, at each iteration, set μ to the last computed Rayleigh quotient.

Algorithm: Rayleigh Quotient Iteration

$v^{(0)} =$ some unit-length vector
$\lambda^{(0)} = r(v^{(0)}) = (v^{(0)})^{\mathsf{T}} A v^{(0)}$
for $k = 1, 2, \ldots$ **do**
  Solve $(A - \lambda^{(k-1)} I) w = v^{(k-1)}$ for w
  $v^{(k)} = w / \|w\|$
  $\lambda^{(k)} = r(v^{(k)}) = (v^{(k)})^{\mathsf{T}} A v^{(k)}$
end for

- The cost per iteration is linear for a tridiagonal matrix.

Convergence of Rayleigh Quotient Iteration

- For real symmetric matrices, we achieve local cubic convergence in Rayleigh quotient iteration (assuming λ_J is a simple eigenvalue)

$$\|v^{(k+1)} - (\pm q_J)\| = \mathcal{O}\left(\|v^{(k)} - (\pm q_J)\|^3\right)$$

and

$$|\lambda^{(k+1)} - \lambda_J| = \mathcal{O}\left(|\lambda^{(k)} - \lambda_J|^3\right).$$

- In other words, each iteration triples the number of digits of accuracy.
- The proof idea is that if $v^{(k)}$ is close to an eigenvector, $\|v^{(k)} - (\pm q_J)\| \leq \epsilon$, then the accuracy of the Rayleigh quotient estimate $\lambda^{(k)}$ is $|\lambda^{(k)} - \lambda_J| = \mathcal{O}\left(\epsilon^2\right)$. One step of inverse iteration then gives

$$\|v^{(k+1)} - q_J\| = \mathcal{O}\left(|\lambda^{(k)} - \lambda_J|\|v^{(k)} - q_J\|\right) = \mathcal{O}\left(\epsilon^3\right).$$

- Rayleigh quotient iteration is great for finding one eigenvalue and its corresponding eigenvector. What if we want to find all eigenvalues?

Operation Counts

In Rayleigh quotient iteration,

- If $A \in \mathbb{R}^{n\times n}$ is a full matrix, then solving $(A-\mu I)w = v^{(k-1)}$ may take $\mathcal{O}\left(n^3\right)$ flops per step.
- If $A \in \mathbb{R}^{n\times n}$ is upper Hessenberg, then each step takes $\mathcal{O}\left(n^2\right)$ flops.
- If $A \in \mathbb{R}^{n\times n}$ is tridiagonal, then each step takes $\mathcal{O}\left(n\right)$ flops.